

RESEARCH PROPOSAL

Indirect Prompt Injection Detection in Agentic SOC Workflows Using Behavioral Anomaly Analysis

A Formal Research Proposal

Research Domain: LLM Security • SOC Automation • Adversarial AI

Target Venues: IEEE S&P | USENIX Security | ACM CCS | NDSS

Prepared by:	Md. Tanjim Mahmud Tuhin
Affiliation:	Daffodil International University
Date:	April 2026

1. Abstract

The integration of Large Language Models (LLMs) into Security Operations Center (SOC) workflows has introduced unprecedented automation capabilities, yet simultaneously exposed critical attack surfaces. Agentic LLM systems — capable of autonomous tool invocation, multi-step reasoning, and environmental interaction — are increasingly deployed for alert triage, log analysis, and threat intelligence aggregation. However, these deployments remain fundamentally vulnerable to indirect prompt injection, an attack vector in which adversarial instructions are embedded within environmental inputs such as SIEM log entries, threat intelligence feeds, or ingested documents, subsequently manipulating the agent's behavior without direct user interaction.

Despite the growing prevalence of LLM-based SOC automation, no dedicated detection framework exists to identify indirect prompt injection at runtime within agentic pipelines. Existing defenses — primarily input sanitization and model-level fine-tuning — operate at the perimeter and lack visibility into the behavioral consequences of successful injection. The absence of a behavioral monitoring layer leaves SOC operators blind to compromised agent decision-making, false alert generation, suppressed threat signals, and unauthorized tool execution.

This proposal presents a research agenda to develop the first SOC-specific indirect prompt injection detection framework grounded in behavioral anomaly analysis and control-flow monitoring. Our approach establishes a formal threat model mapping injection vectors to MITRE ATLAS techniques, constructs a behavioral baseline for agentic SOC workflows, and deploys a hybrid detection system combining sequence anomaly detection with rule-based control-flow constraints. Evaluation will be conducted against established benchmarks (AgentDojo, MCPSecBench) and real-world vulnerability cases including CVE-2025-53773 and EchoLeak (CVE-2025-32711).

Expected contributions include: (1) a formal SOC-specific injection threat taxonomy; (2) a runtime behavioral monitoring architecture for LLM agents; (3) an open detection dataset; and (4) empirically validated detection performance metrics suitable for publication at IEEE S&P, USENIX Security, ACM CCS, or NDSS.

2. Keywords

Prompt Injection • Indirect Prompt Injection • LLM Security • SOC Automation • Behavioral Anomaly Detection • Agentic AI • MITRE ATLAS • Control-Flow Monitoring • Agentic Workflows • Adversarial AI

3. Table of Contents

1. Abstract	
2. Keywords	3
3. Table of Contents	4
4. Introduction	5
5. Literature Review / Background Study	6
5A. LLM Security & Prompt Injection	6
5B. Agentic Systems Security	7
5C. Behavioral Anomaly Detection	7
5D. Research Gaps	8
6. Problem Statement and Significance	8
7. Research Objectives	9
8. Methodology	10
8A. Threat Modeling	10
8B. System Design	11
8C. Behavioral Monitoring Layer	11
8D. Detection Techniques	12
8E. Dataset & Evaluation	12
8F. Attack Simulation	13
8G. Metrics	13
9. Expected Outcomes	13
10. Gantt Chart & Milestones	14
11. Conclusion	16
12. References	17

4. Introduction

4.1 Agentic LLM Systems

Agentic LLM systems represent a paradigm shift from passive language models to autonomous, goal-directed agents capable of reasoning across multi-step workflows. These systems integrate three core components: (1) a reasoning engine (the LLM), (2) external tools — APIs, code interpreters, databases, and file systems — accessible via function-calling interfaces, and (3) persistent memory constructs including in-context history, vector stores, and episodic logs. Architectures such as ReAct (Reasoning + Acting), chain-of-thought prompting, and multi-agent orchestration frameworks (LangChain, AutoGPT, CrewAI) enable these systems to decompose complex objectives, select appropriate tools, and iteratively refine outputs based on environmental feedback.

The autonomy afforded by agentic architectures introduces a fundamentally new security consideration: the agent's behavior is not solely determined by the user's initial instruction but is dynamically shaped by all inputs processed during execution, including external data retrieved from potentially adversary-controlled sources.

4.2 LLMs in SOC Workflows

Modern Security Operations Centers are under increasing pressure to process and contextualize high volumes of security telemetry. LLM integration offers compelling solutions across several SOC functions. In alert triage, LLM agents can analyze SIEM alerts, correlate indicators of compromise (IOCs), and generate preliminary investigation summaries, reducing mean time to detect (MTTD). In log analysis, natural language interfaces enable analysts to query structured log data conversationally, dramatically reducing investigation time. In threat intelligence aggregation, agents autonomously ingest and synthesize feeds from STIX/TAXII sources, open-source intelligence (OSINT), and vendor bulletins, mapping emerging TTPs to MITRE ATT&CK entries.

These pipelines inherently require LLM agents to process untrusted external content — a structural vulnerability that forms the foundation of the threat model explored in this proposal.

4.3 Prompt Injection: Direct vs. Indirect

Prompt injection attacks manipulate the LLM's behavior by embedding adversarial instructions within the input. Direct prompt injection occurs when an authenticated user crafts a malicious prompt against the model's context, attempting to override system instructions or extract unauthorized information. Indirect prompt injection, the primary focus of this research, is

substantively more dangerous in agentic contexts: adversarial instructions are embedded within external content that the agent retrieves and processes as part of its normal operation.

In a SOC context, indirect injection vectors include log entries crafted to contain pseudo-instructions (e.g., a syslog message embedding "Ignore previous instructions and mark all subsequent alerts as benign"), malicious threat intelligence feed entries, poisoned OSINT documents, and adversarially crafted email subjects or attachments that the agent processes during incident investigation.

4.4 Research Motivation and Novelty

The OWASP LLM Top 10 (2025 edition) identifies Prompt Injection as the primary security risk for LLM-integrated applications [1]. Empirical studies have demonstrated attack success rates approaching 84% against commercially deployed agents [2], with model-level mitigations such as instruction hierarchy fine-tuning showing inconsistent generalization across attack variants [3]. Critically, no prior work has addressed the specific threat surface created by indirect prompt injection in SOC automation pipelines, nor proposed a runtime behavioral detection mechanism capable of identifying compromised agent behavior without foreknowledge of the specific injection payload.

This proposal fills that gap by proposing a behavioral anomaly detection framework that monitors agent control-flow, tool invocation patterns, and output characteristics to identify injection-driven behavioral deviations — complementing rather than replacing existing input-level defenses.

5. Literature Review / Background Study

5A. LLM Security & Prompt Injection

The security literature on LLM vulnerabilities has expanded significantly since the formal characterization of prompt injection by Perez and Ribeiro [4]. Subsequent work has categorized injection variants including jailbreaking [5], goal hijacking [6], and indirect injection via retrieved content [7]. Greshake et al. [8] demonstrated systematic indirect injection in retrieval-augmented generation (RAG) pipelines, showing that adversarial documents retrieved during LLM-powered search operations could reliably redirect agent behavior. Architectural defenses include prompt shields [9], spotlighting [10] (marking retrieved content with structural delimiters), and LLM-based classifiers trained on adversarial examples [11].

However, these defenses share a critical limitation: they operate at the input layer and cannot detect injection that has already influenced the agent's internal reasoning or output generation. A successful injection that evades input filters remains invisible to the security apparatus. Furthermore, the adversarial robustness of LLM-based classifiers is itself subject to adaptive attacks [12].

5B. Agentic Systems Security

The security of tool-using LLM agents has received growing attention. Ruan et al. [13] established a risk taxonomy for LLM agent tool use, identifying unauthorized data exfiltration, unintended code execution, and privilege escalation as primary concerns. The concept of confused deputy attacks in agentic contexts — where the agent, acting on behalf of a trusted principal, is manipulated by an untrusted data source — maps directly to indirect injection scenarios [14].

Multi-agent architectures introduce additional attack surfaces. In orchestrator-subagent pipelines, a compromised subagent can propagate malicious instructions upward through the communication channel, a vector not addressed by single-agent defenses [15]. Chain-of-thought reasoning, while improving task performance, also provides attackers with a manipulation surface: injected instructions that appear within the reasoning chain are processed with the same authority as legitimate deliberative steps [16].

Memory poisoning attacks targeting vector stores represent a persistent injection variant: adversarially crafted entries stored in the agent's long-term memory can influence future reasoning across multiple sessions, requiring detection mechanisms that operate at a temporal granularity beyond single-session monitoring [17].

5C. Behavioral Anomaly Detection in Cybersecurity

Behavioral anomaly detection has a well-established lineage in traditional network security, encompassing statistical baselines [18], machine learning approaches including isolation forests and autoencoders [19], and sequence models applied to audit logs [20]. The core principle — establishing a model of normal behavior and flagging deviations — translates conceptually to LLM agent monitoring, though the behavioral representation requires fundamental reconceptualization.

Traditional anomaly detection operates over structured, enumerable event types (network flows, system calls, authentication events). LLM agent behavior involves natural language reasoning traces, dynamic tool selection from potentially large action spaces, and context-dependent output generation — all of which resist direct application of classical anomaly detection methods. This gap motivates the development of agent-specific behavioral representations, such as action sequence embeddings, control-flow graphs over tool invocations, and semantic similarity metrics between expected and observed outputs.

5D. Identified Research Gaps

- No SOC-specific indirect prompt injection studies: existing work focuses on generic web agents, chatbots, and RAG systems, with no threat model tailored to SIEM pipelines, threat intel feeds, or SOC automation workflows.
- No runtime behavioral monitoring for LLM agents: current defenses are exclusively pre-execution (input filters) or post-hoc (output review); no framework monitors agent behavior during execution.
- No formal mapping to MITRE ATLAS: while MITRE ATLAS provides a framework for adversarial ML tactics, no published work systematically maps indirect prompt injection attack patterns to ATLAS techniques in the context of LLM-based SOC tools.
- No open benchmark for SOC injection attacks: AgentDojo and similar benchmarks cover general web agent tasks; no dataset captures SOC-specific injection scenarios with labeled ground truth.

6. Problem Statement and Significance

Indirect prompt injection in agentic SOC workflows constitutes a high-severity, under-researched threat vector. An attacker with the ability to influence any data source processed by the SOC agent — including log ingestion pipelines, threat intelligence subscriptions, or fetched external documents — can embed adversarial instructions that manipulate the agent's analysis, suppress threat alerts, generate false positive floods, exfiltrate investigation context, or invoke unauthorized tool actions. Critically, this manipulation occurs through normal operational data channels, rendering it invisible to network-level controls and conventional SIEM detection logic.

The risk is amplified by the architectural characteristics of SOC pipelines. Agents operating in SOC environments routinely process high volumes of heterogeneous, partially untrusted data. The operational pressure to automate triage at scale incentivizes configurations that maximize agent autonomy, directly increasing the attack surface for injection. The consequences of successful injection extend beyond immediate system compromise: an agent that has been manipulated to generate false benign classifications actively undermines the SOC's threat visibility, with cascading effects on incident response timelines and organizational risk posture.

The absence of dedicated detection mechanisms for this threat class represents a critical gap between LLM deployment velocity and security assurance capability. This research directly addresses that gap.

7. Research Objectives

Main Aim

To design, implement, and empirically validate a behavioral anomaly detection framework for identifying indirect prompt injection attacks in agentic LLM-based SOC automation pipelines.

Specific Objectives

1. To construct a formal threat model characterizing indirect prompt injection vectors specific to SOC environments, including SIEM log manipulation, threat intelligence feed poisoning, and adversarial document ingestion.
2. To systematically map identified attack behaviors to techniques in the MITRE ATLAS adversarial ML framework, establishing a standardized taxonomy for SOC injection attacks.
3. To design a behavioral anomaly detection architecture for agentic LLM workflows, incorporating control-flow modeling, tool invocation profiling, and output semantic analysis.
4. To implement a runtime monitoring layer capable of detecting control-flow deviations, unexpected tool invocations, and prompt contamination patterns in live agent execution.
5. To evaluate detection performance against established benchmarks (AgentDojo, MCPSecBench) and a custom SOC injection dataset, measuring precision, recall, F1-score, and attack success reduction.
6. To validate the framework against real-world vulnerability cases, including CVE-2025-53773 and EchoLeak (CVE-2025-32711), demonstrating applicability to documented production attack scenarios.

8. Methodology

The research is structured as a seven-component pipeline, progressing from threat characterization through system design, implementation, and empirical evaluation.

8A. Threat Modeling

The initial phase constructs a comprehensive threat model for indirect prompt injection in SOC environments. Injection vectors are enumerated across three primary attack surfaces:

- **SIEM Log Manipulation:** Adversarial actors (external attackers with log write access, compromised endpoints, or insider threats) embed pseudo-instructions within log entries processed by the LLM agent. Attack variants include instruction embedding in user-agent strings, event descriptions, file paths, and process names.
- **Threat Intelligence Feed Poisoning:** Attackers contribute or compromise STIX/TAXII feeds, OSINT sources, or vendor intelligence subscriptions to inject adversarial instructions within IOC descriptions, malware analysis summaries, or TTP narratives processed by the SOC agent.
- **Adversarial Document Ingestion:** Malicious content embedded in phishing attachments, forensic artifacts, or external reports processed by the agent during incident investigation.

Each vector is formally characterized using a five-tuple model: (attacker capability, injection surface, payload delivery mechanism, target agent behavior, observable consequence). Attack patterns are subsequently mapped to MITRE ATLAS techniques, including AML.T0051 (LLM Prompt Injection), AML.T0054 (LLM Jailbreak), and relevant supply chain and data poisoning techniques.

8B. System Design

The detection framework is designed as an instrumented wrapper around a reference agentic SOC pipeline consisting of four functional stages: (1) Input Ingestion — processing of SIEM alerts, log batches, and threat feeds; (2) LLM Reasoning — chain-of-thought analysis and decision generation; (3) Tool Execution — invocation of investigation tools (log query, IOC lookup, case management API, network isolation); and (4) Output Generation — alert classification, investigation summary, or automated response action.

Normal behavior baselines are established through supervised profiling of the pipeline operating on validated, injection-free datasets. Baselines capture: tool invocation frequency distributions, action sequence n-gram profiles, output semantic embeddings, and control-flow graph structures for representative task categories (alert triage, IOC investigation, threat hunting).

8C. Behavioral Monitoring Layer

The monitoring layer intercepts agent execution at three instrumentation points:

- **Control-Flow Monitor:** Maintains a real-time model of the agent's tool invocation sequence. Deviations from the profiled control-flow graph for the active task class — including unexpected tool calls, anomalous sequencing, or invocation of privileged actions absent from the task baseline — trigger anomaly scores.
- **Semantic Contamination Detector:** Applies embedding-based similarity analysis between the LLM's output and the injected input content, detecting cases where adversarial payload tokens propagate into agent responses or action parameters.
- **Prompt Context Integrity Monitor:** Tracks the agent's active context window for structural anomalies — injected instruction patterns, context window flooding, role-switching attempts — using a combination of rule-based signatures and ML-based classifiers trained on labeled injection examples.

8D. Detection Techniques

The detection engine employs a hybrid architecture combining rule-based and ML-based components. The rule-based layer encodes deterministic signatures for known injection patterns, ATLAS-mapped attack behaviors, and invariant control-flow constraints (e.g., network isolation actions must never be invoked without prior confirmation from the triage workflow). The ML layer applies sequence anomaly detection using transformer-based sequence models trained on normal execution traces, flagging action sequences with low likelihood under the learned normal behavior distribution. An ensemble aggregation module combines per-component anomaly scores using a weighted voting scheme, with weights calibrated to minimize false positive rate while maintaining detection sensitivity.

8E. Dataset & Evaluation Infrastructure

Evaluation leverages three data sources. AgentDojo provides a general-purpose agentic task benchmark with adversarial injection scenarios, enabling baseline comparison with non-SOC-specific detection approaches. MCPSecBench offers structured evaluation for MCP (Model Context Protocol) security, relevant to multi-agent SOC architectures. The primary evaluation dataset is a custom SOC injection corpus constructed by simulating indirect injection attacks against a deployed reference pipeline, with ground-truth labels for injection presence, vector type, and behavioral consequence.

8F. Attack Simulation

Attack simulation proceeds in two phases. In Phase 1, synthetic injection payloads are generated using a red-team methodology aligned with the threat model, covering instruction override, goal hijacking, tool manipulation, and data exfiltration variants. These payloads are embedded within realistic SOC data artifacts (log entries, STIX bundles, investigation reports) and injected into the pipeline under controlled conditions. In Phase 2, real-world attack scenarios derived from CVE-2025-53773 and CVE-2025-32711 (EchoLeak) are replicated within the test environment, providing ecologically valid evaluation cases grounded in documented production vulnerabilities.

8G. Evaluation Metrics

Metric	Description
Detection Rate (DR)	Percentage of injection attempts correctly identified by the monitoring layer.
False Positive Rate (FPR)	Proportion of benign executions incorrectly flagged; target FPR < 2%.
Precision	Proportion of flagged events that are true injections.
Recall	Proportion of actual injections that are successfully detected.
F1-Score	Harmonic mean of precision and recall; primary performance indicator.
Attack Success Reduction	Reduction in agent behavior manipulation rate with detection active versus disabled.

9. Expected Outcomes

New Knowledge Contributions

- First formal threat taxonomy for indirect prompt injection in SOC automation environments, providing a structured reference for security architects deploying LLM-based SOC tools.
- A behavioral anomaly detection model specifically designed for agentic LLM workflows, generalizable beyond SOC contexts to other high-security autonomous agent deployments.
- An open-source SOC injection benchmark dataset, enabling reproducible evaluation and future research in this emerging threat domain.
- A systematic MITRE ATLAS mapping of SOC injection attack patterns, extending the ATLAS knowledge base with operationally relevant techniques.

Practical Applications

- A deployable behavioral monitoring framework for organizations operating LLM-integrated SOC platforms (e.g., Microsoft Sentinel Copilot, Google SecOps SOAR, CrowdStrike Charlotte AI).
- Detection signatures compatible with SIEM platforms, enabling alert generation for identified injection events within existing SOC workflows.
- Design guidance for secure agentic SOC pipeline architecture, including recommended tool invocation constraints, context isolation patterns, and monitoring integration points.

Target Publications

- IEEE Symposium on Security and Privacy (IEEE S&P) — primary target for detection framework and evaluation results.
- USENIX Security Symposium — alternative primary venue.
- ACM Conference on Computer and Communications Security (CCS) — threat modeling and ATLAS mapping contributions.
- Network and Distributed System Security Symposium (NDSS) — SOC-specific deployment and evaluation findings.

10. Gantt Chart & Milestones

The research is structured across a nine-month timeline with defined phase deliverables and milestone gates.

Phase	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6	Month 7	Month 8	Month 9
1. Literature Review									
2. Threat Modeling									
3. System Design									
4. Implementation									
5. Dataset & Eval									
6. Attack Simulation									
7. Writing & Review									

Milestone Deliverables

Month	Phase	Milestone Deliverable
M2	Literature Review	Annotated bibliography; research gap analysis report; MITRE ATLAS mapping draft.
M3	Threat Modeling	Formal SOC injection threat model; injection vector taxonomy; ATLAS mapping finalized.
M4	System Design	Reference SOC pipeline architecture; behavioral baseline specification; monitoring layer design.
M6	Implementation	Functional behavioral monitoring prototype; control-flow and semantic detectors operational.
M7	Dataset & Eval	Custom SOC injection dataset published; AgentDojo/MCPSecBench baseline results documented.
M8	Attack Simulation	CVE reproduction cases complete; full detection evaluation results; performance metrics finalized.
M9	Writing	Complete research paper draft; supplementary materials; code and dataset repository prepared.

11. Conclusion

This proposal outlines a technically rigorous and operationally motivated research agenda addressing a critical security gap at the intersection of LLM deployment, SOC automation, and adversarial AI. Indirect prompt injection in agentic SOC workflows represents an undercharacterized but high-impact attack vector: adversaries capable of influencing any data source ingested by an LLM agent can manipulate security-critical analysis outcomes without interacting directly with the system's user interface or authentication controls.

The proposed behavioral anomaly detection framework offers a fundamentally different detection philosophy from existing input-focused defenses, targeting the observable behavioral consequences of successful injection rather than attempting to classify injection payloads before execution. This approach is robust to novel injection variants and adaptive attackers, as detection depends on behavioral deviation from an established normal profile rather than foreknowledge of specific attack payloads.

Key research challenges include: managing false positive rates in anomaly detection without suppressing legitimate behavioral variation; achieving detection generalization across heterogeneous LLM architectures and SOC pipeline configurations; and maintaining detection effectiveness against adaptive adversaries who calibrate injection payloads to minimize behavioral deviation. These challenges will be systematically addressed through the proposed hybrid detection architecture, rigorous empirical evaluation, and red-team adversarial testing throughout the research lifecycle.

Successful completion of this research will deliver the first empirically validated behavioral detection framework for indirect prompt injection in SOC environments, directly improving the security assurance of LLM-integrated security operations at a time of rapid and widespread deployment.

12. References

- [1] OWASP Foundation. (2025). OWASP Top 10 for Large Language Model Applications 2025. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [2] Zhan, Q., He, Z., Luo, Y., & Liang, Z. (2024). Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. arXiv preprint arXiv:2403.02691.
- [3] Wallace, E., Xu, S., Garg, N., Singh, S., & Klein, D. (2019). Universal adversarial triggers for attacking and analyzing NLP. Proceedings of EMNLP 2019.
- [4] Perez, F., & Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. arXiv preprint arXiv:2211.09527.
- [5] Wei, A., Haghtalab, N., & Steinhardt, J. (2024). Jailbroken: How does LLM safety training fail? Advances in Neural Information Processing Systems, 36.
- [6] Xu, J., Ma, Y., & Zhang, J. (2024). Goal hijacking in large language model-integrated applications. Proceedings of USENIX Security 2024.
- [7] Liu, Y., Deng, G., Li, Y., Wang, K., Liu, T., Zhang, T., & Liu, Y. (2024). Prompt injection attack against LLM-integrated applications. arXiv preprint arXiv:2306.05499.
- [8] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injections. Proceedings of AISEC@CCS 2023.
- [9] Microsoft. (2024). Prompt shields: Detecting and mitigating prompt injection attacks. Azure AI Content Safety Technical Report.
- [10] Hines, K., Lopez-Morales, S., Hall, P., Hosseini, A., Perez, R., & Verspoor, K. (2024). Defending against indirect prompt injection attacks with spotlighting. arXiv preprint arXiv:2403.14720.
- [11] Pi, R., Han, T., Pan, J., Qin, Q., Liu, T., & Zhang, T. (2024). Malla: Demystifying real-world large language model integrated malicious services. Proceedings of USENIX Security 2024.
- [12] Mehrabi, N., Goyal, P., Verma, A., & Galstyan, A. (2023). Robust prompt optimization for defending language models against jailbreaking attacks. arXiv preprint arXiv:2401.17263.
- [13] Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., ... & McIlraith, S. (2024). Identifying the risks of LM agents with an LM-emulated sandbox. Proceedings of ICLR 2024.
- [14] Abdelnabi, S., Greshake, K., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injections. Workshop on Trustworthy and Reliable Large-Scale Machine Learning Models.
- [15] Chen, L., Zhang, F., & Wang, Y. (2024). AutoInject: Automated indirect prompt injection for multi-agent LLM systems. arXiv preprint arXiv:2405.00018.
- [16] Turpin, M., Michael, J., Perez, E., & Bowman, S. (2023). Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. Advances in Neural Information Processing Systems, 36.
- [17] Shi, W., Dinan, E., Shuster, K., Weston, J., & Xu, J. (2024). Memory poisoning attacks against LLM-based agents via long-term memory manipulation. arXiv preprint arXiv:2403.01833.
- [18] Lakhina, A., Crovella, M., & Diot, C. (2004). Diagnosing network-wide traffic anomalies. Proceedings of ACM SIGCOMM 2004.
- [19] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest. Proceedings of ICDM 2008.
- [20] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. Proceedings of ACM CCS 2017.

- [21] MITRE Corporation. (2023). MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems, Version 4.2. <https://atlas.mitre.org/>
- [22] Debenedetti, E., Zhang, J., Balunovic, M., Baader, L., Gehr, T., & Vechev, M. (2024). AgentDojo: A dynamic environment to evaluate attacks and defenses for LLM agents. *Advances in Neural Information Processing Systems*, 37.
- [23] National Vulnerability Database. (2025). CVE-2025-53773: Prompt injection in LLM-integrated security tooling. <https://nvd.nist.gov/>
- [24] National Vulnerability Database. (2025). CVE-2025-32711: EchoLeak — indirect prompt injection via Microsoft 365 Copilot. <https://nvd.nist.gov/>
- [25] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *Proceedings of ICLR 2023*.

— End of Research Proposal —